Embeddings, Similarity, CLIP

Sattam Altwaim
satttam.tuuaim@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology



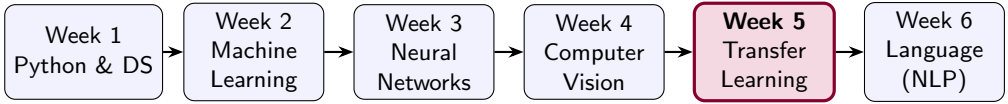
KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 5, Day 31, Lecture

19 October 2026

Table of Contents

1. Big Picture and Motivation
2. Learning Outcomes
3. Embeddings: Photos as Points in a Space
4. Search, kNN and the Probe on Embeddings
5. CLIP: One Space for Photos and Sentences
6. Olympiad Practice
7. Limitations and What Comes Next
8. Summary
9. References

Where We Are



Today: Day 31 of 42

- **Days 29 and 30:** a pretrained encoder turns a photo into 512 numbers; a head names classes from them; we trained the head, then let stages of the encoder move. Every task so far ended in a **label**.
- **Today:** the 512 numbers themselves. As an **embedding** they are a point in a space where similar photos sit close together: **similarity** and **search** without any training, **kNN** and the **linear probe** on cached embeddings, and **CLIP**, whose space holds photos *and sentences*: classification from a prompt, **zero-shot**, and text-to-image **retrieval**.
- **This week:** Day 32 freezing step by step; Day 33 embedding models, linear probing, CLIP zero-shot and retrieval in depth; Day 34 the small-data habits; Friday the competition. Week 6 does the same with text.

How We Did It Before: Comparing Photos by Their Pixels

- **The task that has no label:** "find the photos most like this one", "is this the same restroom icon", "which of these 400 photos shows a red car". Nothing to train a head on.
- **Week 4's tool was the pixel tensor.** A photo is $3 \times 224 \times 224 = 150{,}528$ numbers; the Euclidean distance between two tensors (Day 02) says how far they are. Nearest photo: smallest distance.
- **Day 29 hinted at the better way:** two daisies gave similar 512-number vectors, a tulip a different one (Lab 1, Task 8). That vector is today's object.

Pixel distance, done by hand

Two photos of the same dog, one shifted 10 pixels to the right: every pixel now lands on a different pixel of the other photo, the distance is large. A dog on grass and a cat on grass: most pixels are green in both, the distance is small. Pixels measure *position and colour*, not *what is in the photo*.

Where It Breaks: Nearest by Pixels Against Nearest by Embedding



Source: Kaggle *Natural Images* (`prasunroy/natural-images`), 50 photos per class, the published ResNet-18 encoder applied once; `scripts/day31_embedding_figures.py`. Green: same class as the query; red: another class.

The problem

Pixels find photos with the same colours and layout, whatever they show. The encoder's 512 numbers find dogs. On 400 photos of eight kinds, the nearest photo by pixels has the right class 46.8% of the time; by embedding, 99.2%.

Today's Idea

In one sentence

An **embedding** places an input in a space where *similar things are close*; a pretrained encoder gives one for free, **cosine similarity** measures closeness, and **CLIP** learned one space for photos and sentences, so a sentence can stand in for a class.

- **Similarity and search:** compare embeddings, not pixels; the nearest neighbours are the search result; a vote among them (kNN) classifies with no training.
- **Cached embeddings:** compute them once, then train a linear probe in seconds (Day 30's probe without the encoder in the loop).
- **CLIP:** an image encoder and a text encoder trained on 400 million (photo, caption) pairs to agree; "a photo of a dog" lands next to dog photos: **zero-shot classification**, **text retrieval**.

Where you will meet it

IOAI 2025, *Restroom Icon Matching*: match each cropped icon to its partner from the same restroom; the organisers' baseline is CLIP embeddings and cosine similarity (today's practice task). Today's labs: search and zero-shot on Kaggle photos; 100 sports with ten labels each.

⇒ Learning Outcomes

By the end of this session, you will be able to:

- **Explain** what an embedding is, why a pretrained encoder's output is one, and what "close" means in that space.
- **Compute** Euclidean distance, dot product and cosine similarity between embeddings by hand on small vectors and with one matrix product in PyTorch.
- **Build** a nearest-neighbour search, a kNN classifier and a linear probe on cached embeddings, and say when each is the right tool.
- **Explain** how CLIP was trained (two encoders, one shared space, the contrastive loss) and why a sentence can replace a label.
- **Run** CLIP zero-shot classification with prompts and text-to-image retrieval with `transformers`, and read the published numbers on where it wins and where it fails.
- **Recognise** a similarity, matching or zero-shot task in an Olympiad statement and plan the solution: embed, normalise, compare.

Embeddings: **Photos as Points in a Space**

Definition

Embedding

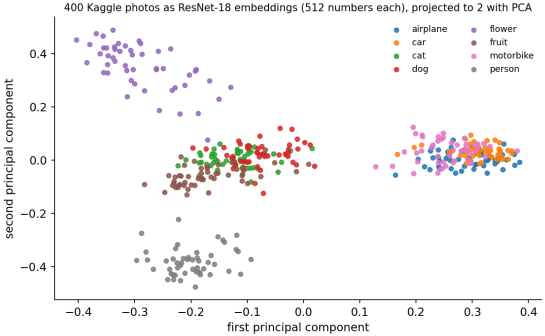
An **embedding** is a fixed-length list of numbers (a **vector**) that represents an input so that *similar inputs get similar vectors*. The space of all such vectors is the **embedding space**; the length of the vector is its **dimension** (d). Day 29's 512 numbers from `avgpool` are a 512-dimensional embedding of the photo; the model producing them is an **embedding model**.

- **Why the encoder's output qualifies:** it was trained so that a *linear* head could separate 1,000 classes from it (Day 29). Photos of the same class must therefore land in the same region; that is the "similar inputs, similar vectors" property, learned as a by-product.
- **Not to be confused with** the head's *scores*: 1,000 numbers that name ImageNet's classes. The embedding sits one layer earlier and knows no class names.

Analogy

A library shelves books by subject: two books on volcanoes stand side by side, a cookbook far away. The shelf position is the book's embedding; "books like this one" are its neighbours, found without reading any of them.

The Picture: 400 Photos, 512 Numbers Each, Drawn in Two



Source: Kaggle *Natural Images*, 50 photos per class, the published ResNet-18 encoder applied once; the 512 numbers of each photo projected to 2 with PCA (Day 13); `scripts/day31_embedding_figures.py`.

- Nobody told the encoder about these eight classes, and no training happened here: the eight clouds come from ImageNet's training alone. Two of 512 dimensions already separate most of them; the other 510 separate the rest.

The Math: Three Ways to Say "Close"

For embeddings $a, b \in \mathbb{R}^d$ with entries a_i, b_i:

$$\text{Euclidean distance} \quad \|a - b\| = \sqrt{\textstyle\sum_i (a_i - b_i)^2}$$

$$\text{dot product} \quad a \cdot b = \textstyle\sum_i a_i b_i$$

$$\text{cosine similarity} \quad \cos(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|} \in [-1, 1]$$

$\|a\| = \sqrt{a \cdot a}$ is the vector's length. Cosine similarity is the dot product of the two vectors after each is divided by its length: the **angle** between them, blind to their size. 1: same direction; 0: unrelated; -1: opposite.

With numbers in three dimensions

$a = (2, 1, 0)$, $b = (4, 2, 0)$, $c = (0, 1, 2)$.

- $\|a - b\| = \sqrt{4 + 1 + 0} = 2.24$, $\|a - c\| = \sqrt{4 + 0 + 4} = 2.83$.
- $a \cdot b = 8 + 2 + 0 = 10$; $\|a\| = \sqrt{5}$, $\|b\| = \sqrt{20}$; $\cos(a, b) = 10/(2.24 \cdot 4.47) = \mathbf{1.00}$.
- $a \cdot c = 1$; $\|c\| = \sqrt{5}$; $\cos(a, c) = 1/5 = \mathbf{0.20}$.

b is a doubled: far by distance, identical by cosine. A brighter copy of a photo is a longer vector in the same direction: hence cosine.

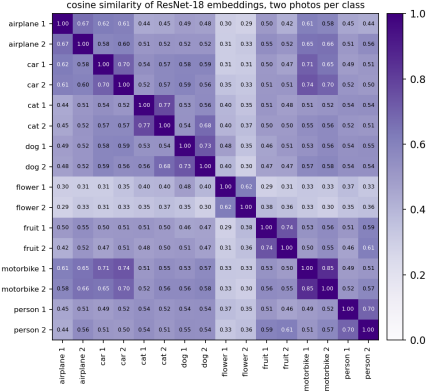
In Code: Embed Once, Normalise, One Matrix Product

```
encoder = resnet18(weights=ResNet18_Weights.DEFAULT).eval()
encoder.fc = nn.Identity()                          # Day 29: keep the 512 numbers
with torch.no_grad():
    E = torch.cat([encoder(xb) for xb, _ in loader])  # (N, 512): one row per photo

E = F.normalize(E, dim=1)       # each row divided by its length: every row now has length
    1
S = E @ E.T                     # (N, N): S[i, j] = cos(photo i, photo j), dot products of
     unit rows
S[i].topk(6)                    # row i's 6 largest: itself (1.0), then its 5 nearest
    neighbours
```

- `F.normalize` is `torch.nn.functional.normalize`: $a/\|a\|$ row by row. After it, a dot product *is* a cosine similarity, so the whole $N \times N$ table is one `@` (Day 05's matrix product). For 400 photos: a 400×400 table in a millisecond.
- `torch.cdist(E, E)` gives Euclidean distances instead; `(E @ E.T)` without normalising gives dot products. Cosine is the common practice for encoder embeddings; the practice task's baseline does exactly these lines.
- Keep `E` on disk (`torch.save`) once computed: the encoder runs once, every later step is table arithmetic.

The Similarity Table of 16 Photos



Source: the same 400 Kaggle photos, two per class, ResNet-18 embeddings; `scripts/day31_embedding_figures.py`.

- The diagonal is 1.00: a photo against itself. Each 2×2 block on the diagonal is a class against itself: 0.72 on average; off the blocks 0.49.
- Two dogs: 0.73. A dog and an airplane: 0.49. A car and a motorbike, two kinds of vehicle: 0.71: the space also knows what is *somewhat* alike.
- No number here is negative or near -1: these embeddings leave a ReLU (Day 29), so every entry is positive and the usable range is narrower than $[-1, 1]$. CLIP's embeddings, later, use the whole range.

Search, kNN and the Probe **on Embeddings**

Nearest-Neighbour Search: Retrieval

image search: the five photos whose CLIP embedding is closest to the query's (cosine similarity)



Source: the 400 Kaggle photos, CLIP ViT-B/32 image embeddings (the model of the next section), cosine similarity; `scripts/day31_embedding_figures.py`.

Definition

Retrieval (image search): given a **query** embedding, return the **gallery** items most similar to it. The gallery is embedded once; a query costs one dot product per item and one `topk`.

- No labels, no training: the neighbours are motorbikes because the space was built that way; a side view finds side views first, since the embedding keeps pose and setting too.
- Scored when labels exist: **precision at** k (how many of the k neighbours share the query's class) or **recall at** k (is the right item among the top k; CLIP reports R@1, R@5, R@10).

kNN on Embeddings: A Classifier With No Training

Definition

k-nearest-neighbour classification (Day 10) on embeddings: embed the labelled photos once; for a new photo, find its k most similar labelled photos and take the majority label. $k = 1$: copy the nearest label.

```
S = E_new @ E_lab.T                  # both
    normalised
top = S.topk(k, dim=1).indices       # (N_new, k
    )
votes = y_lab[top]                   # the k
    labels
pred = votes.mode(dim=1).values      # majority
    vote
```

- **Measured on the 400 Kaggle photos**, each photo classified by its nearest *other* photo (leave-one-out, $k = 1$): pixels 46.8%, ResNet-18 embeddings 99.2%, CLIP embeddings 100%. Same photos, same rule; only the space changed.
- **When it shines:** few labelled photos per class, a new class added in seconds (embed its photos, append them), no GPU at prediction time beyond the encoder.
- **When it fails:** a gallery of millions (every query touches all of it; libraries such as FAISS index it), and classes whose photos vary more than the space separates them.

The Linear Probe on Cached Embeddings

```
class Probe(nn.Module):                  # Day
    30's linear probe
    def __init__(self, d, k):
        super().__init__()
        self.fc = nn.Linear(d, k)
    def forward(self, e):
        return self.fc(e)                #
            scores from an embedding

probe = Probe(512, 100)
opt = torch.optim.Adam(probe.parameters(), lr
    =1e-2)
for epoch in range(100):      # 1,000 cached
    rows: seconds
    for eb, yb in loader:     # batches of
        embeddings
        loss = criterion(probe(eb), yb)
        opt.zero_grad(); loss.backward(); opt.
            step()
```

- **Same probe as Day 30**, one change: the encoder is *outside* the loop. Embed every photo once (`E`), and the loop only multiplies a $512 \times k$ matrix: 100 epochs in seconds instead of minutes, and the encoder never needs a GPU again.
- That is what "logistic regression on fixed features" meant in Kornblith's table and Kumar's "LP". scikit-learn's `LogisticRegression` on `E.numpy()` does the same job in one line.
- **Cost of the shortcut:** no augmentation (a photo's embedding is fixed) and the encoder cannot adapt: the price Day 30 measured.

Which Tool When

Tool	Needs	Training	Use when	Reads as
retrieval (nearest neighbours)	a gallery, a query	none	"find the most similar", matching, duplicates, search	cosine, `topk`
kNN on embeddings	a few labelled photos per class	none	tiny labelled sets, classes that change	majority of k neighbours
linear probe on cached embeddings	tens to hundreds per class	seconds	most labelled tasks; the first number to get	Day 30's probe, encoder outside the loop
CLIP zero-shot	class *names* only	none	no labelled photos at all; a quick baseline	prompts, next section
fine-tuning	hundreds per class, a GPU	minutes to hours	the probe is far from the goal, photos far from ImageNet	Day 30

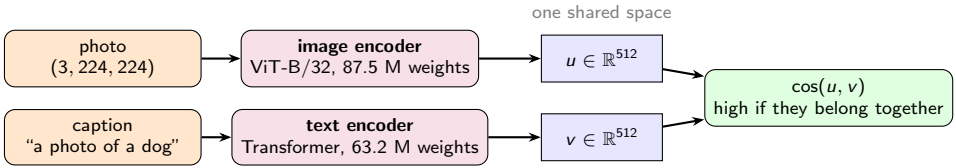
- In a contest, run the first four in that order on the same cached embeddings: each costs seconds, and the validation accuracy of the probe tells you whether fine-tuning is worth the GPU minutes.
- Every row starts with the same two lines: embed, normalise. Day 33 compares embedding models for the first three rows; Week 6 does all of this with sentences.

CLIP: **One Space for Photos and Sentences**

What CLIP Is

Definition

CLIP (Contrastive Language-Image Pre-training, Radford et al., 2021): an image encoder and a text encoder trained on **400 million (photo, caption) pairs** from the internet so that a photo and its own caption get *similar* embeddings and a photo and another caption *dissimilar* ones, in one 512-dimensional space.



- The image encoder is a **Vision Transformer** (Day 29's table) or a ResNet, the text encoder a Transformer over the caption's tokens; each ends in a `Linear` projection to 512 numbers.
- Trained in batches of **32,768** pairs for 32 epochs. We only *load* it: `openai/clip-vit-base-patch32`, 151 million weights.

The Mechanism: Contrastive Training on an $N \times N$ Table

	v_1	v_2	v_3	v_4
u_1	s_{11}	s_{12}	s_{13}	s_{14}
u_2	s_{21}	s_{22}	s_{23}	s_{24}
u_3	s_{31}	s_{32}	s_{33}	s_{34}
u_4	s_{41}	s_{42}	s_{43}	s_{44}

rows: N photos of a batch; columns: their captions; $s_{ij} = \cos(u_i, v_j)/\tau$; purple: true pairs

The loss

Row i is a classification with N "classes": which caption belongs to photo i? Softmax over the row, cross-entropy against column i; the same for every column. Average the $2N$ losses, step on *both* encoders.

With tiny numbers

$N = 2$, $\tau = 1$. Row 1: $s_{11} = 0.9$ (the dog and its caption), $s_{12} = 0.1$ (the dog and "a red car"). Softmax: $p_{11} = e^{0.9}/(e^{0.9} + e^{0.1}) = 2.46/3.56 = 0.69$; loss $= -\ln 0.69 = 0.37$. Training pushes s_{11} up and s_{12} down until $p_{11} \to 1$: the photo moves towards its own caption, away from every other. τ is a learned temperature; the published model has $1/\tau = 100$, so logits are $100\times$ cosine.

Zero-Shot Classification: A Sentence Instead of a Label

Definition

Zero-shot classification: classify photos into classes the model never saw labelled, using only the class *names*. With CLIP: write one sentence per class ("a photo of a {class}"), embed the sentences once, embed the photo, and pick the sentence with the highest cosine similarity. Zero labelled photos, zero training.

1. **Prompts:** k sentences $\to$ text embeddings $v_1, \ldots, v_k$, normalised. This is the "head": a $k \times 512$ matrix *written by the text encoder* instead of learned from photos.
2. **Photo** $\to$ image embedding u, normalised.
3. **Scores** $z_j = 100 \cdot \cos(u, v_j)$; **probabilities** $p = \mathrm{softmax}(z)$; the class is $\arg\max_j p_j$.

- Read step 3 next to Day 29's head: $z = Wu$ with W the stacked prompt embeddings. The paper says it: the text encoder "generates the weights of a linear classifier based on the text". A probe trains W from photos; zero-shot writes it from words.
- **Published:** zero-shot CLIP (ViT-L/14 at 336 px) reaches **76.2%** top-1 on ImageNet, the original supervised ResNet-50's accuracy, with none of its 1.28 million labels. Our small ViT-B/32: 63.2%.

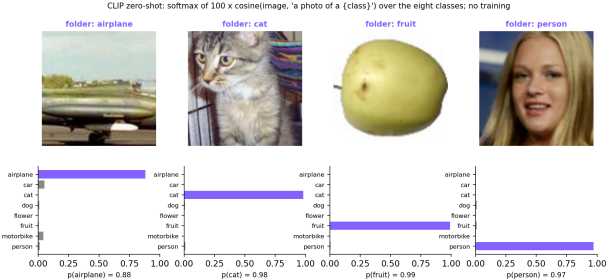
In Code: CLIP Zero-Shot With transformers

```python
from transformers import CLIPModel, CLIPProcessor
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32").eval()
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

prompts = [f"a photo of a {c}" for c in CLASSES]              # one sentence per class
text_in = processor(text=prompts, return_tensors="pt", padding=True)   # tokens
image_in = processor(images=photos, return_tensors="pt")      # (N, 3, 224, 224), CLIP's
    mean and std
with torch.no_grad():
    T = model.get_text_features(**text_in)                    # (k, 512)
    U = model.get_image_features(**image_in)                  # (N, 512)
T = F.normalize(getattr(T, "pooler_output", T), dim=1)        # transformers 5 wraps the
    tensor
U = F.normalize(getattr(U, "pooler_output", U), dim=1)        # unit length: dot = cosine
probs = (100 * U @ T.T).softmax(dim=1)                        # (N, k): one row per photo
```

- Day 29's three steps again: the processor resizes, crops and normalises photos with CLIP's own mean and std, and tokenises text; `get_image_features` and `get_text_features` are the two encoders with their projections.
- `model(**processor(text=..., images=...))` does all of it at once and returns `logits_per_image`; the explicit form caches `T` once, as the paper recommends. IOAI 2025's baseline wrote the same lines with OpenAI's `clip` package.

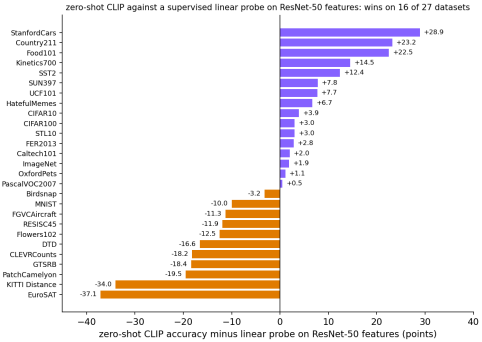
Zero-Shot on the Kaggle Photos, and the Prompt



Source: Kaggle *Natural Images*, CLIP ViT-B/32 applied once with the prompt "a photo of a {class}"; `scripts/day31_embedding_figures.py`.

- On the 400 photos: 99.8% with "a photo of a {class}", 98.0% with the bare class names. **Prompt engineering** (Radford et al., 2021): "A photo of a {label}." alone adds 1.3 points on ImageNet; a task hint helps more ("a photo of a {label}, a type of pet", "a satellite photo of a {label}"); averaging the embeddings of 80 prompt variants adds 3.5 more: almost 5 points together, "free" once the text embeddings are cached.

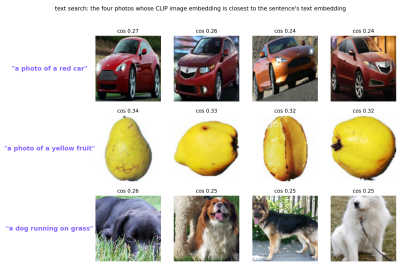
The Evidence: Where Zero-Shot CLIP Wins and Fails



Source: Radford et al. (2021), Figure 5: zero-shot CLIP minus a supervised logistic regression on ResNet-50 features; `scripts/day31_embedding_figures.py`.

- **Wins** where the internet has captions: cars +28.9, food +22.5, actions in video +14.5; general object sets a few points (ImageNet +1.9).
- **Fails** on what captions never describe: satellite tiles (−37.1), lymph-node slides (−19.5), traffic signs (−18.4), counting, digits, fine-grained flowers and aircraft.
- **Against few labels** (Figure 6): zero-shot CLIP equals a 4-shot probe on its own embeddings on average and a 16-shot probe on the best other encoders. With 16 or more labels per class, the probe wins.

Retrieval With Text: The Query Is a Sentence



Source: the 400 Kaggle photos, CLIP ViT-B/32, cosine similarity between each sentence's text embedding and the photos' image embeddings; `scripts/day31_embedding_figures.py`.

- The same `topk` as image search, with a text embedding as the query: colour, action and setting are searchable because captions mention them. Nothing in the gallery was labelled "red" or "running".
- **Published:** on Flickr30k, zero-shot CLIP finds the right photo for a caption in its top 10 **95.2%** of the time (R@10), and the right caption for a photo 99.4% (Radford et al., 2021, Table 13); fine-tuned systems of the time reached 96.8% and 99.2%.

Embeddings: **Olympiad Practice**

Practice Task: Match the Restroom Icons

Task

Restroom signs come in pairs: a male icon and a female icon drawn in the same style. You get a training folder of 82 restrooms, four images each (the male and female icons, as the original sign and as a tight crop), and a test set with a **query** folder of cropped icons and a **gallery** of original signs, two per query (the two icons of its restroom), file names shuffled. For every query, name the gallery image that is the *opposite-gender icon of the same restroom*. Score: the fraction of correct matches. 30 queries, one GPU, a time limit.

- **Question 1:** which of today's tools is this, and what are the query and the gallery in embedding terms?
- **Question 2:** the nearest gallery image by cosine is usually the *same-gender* icon of the same restroom (same style). How do you make the rule return the opposite one?
- **Question 3:** 82 training restrooms are few. Name one use of them without fine-tuning, and one with.

Source: IOAI 2025 (Beijing), Individual Contest, Task 4, "Restroom Icon Matching" (`github.com/IOAI-official/IOAI-2025`). The organisers' baseline, CLIP ViT-B/16 embeddings and cosine similarity, scored 0.77 on the final leaderboard; the committee's best solution 0.90.

Solution Idea and Code

Recognise: retrieval with a twist: the nearest neighbour is the wrong answer by design, so the rule needs one more step.

```
Q = F.normalize(embed(query_paths), dim=1)          # (30, 512), cached once
G = F.normalize(embed(gallery_paths), dim=1)        # (60, 512)
S = Q @ G.T                                          # (30, 60): every query against the
    gallery
g_gender = zero_shot(G, ["a male icon", "a female icon"])   # (60,)
for i in range(len(Q)):
    S[i, g_gender == q_gender[i]] = -1                # forbid the query's own gender
    match[i] = S[i].argmax()                          # the most similar icon of the other
        gender
```

1. **Embed, normalise, one matrix product**: queries against the gallery; the baseline stops at `argmax` (0.77).
2. **Use the structure:** each query has two partners in the gallery, one of each gender. Zero-shot CLIP tells male from female icons with two prompts; mask the query's own gender, then take the best remaining.
3. **The 82 training restrooms:** without fine-tuning, check the rule on them and tune the prompts; with it, Day 30's recipe on the image encoder with a loss that pulls each crop to its opposite-gender original. Validate on held-out restrooms.

Embeddings: **Limitations & What Comes Next**

Limitations

- **The space is only as good as the training data.** ImageNet's encoder places photos by what ImageNet cared about; CLIP's by what internet captions mention. Satellite tiles, slides, digits and traffic signs are nearly unseen: zero-shot CLIP loses 10 to 37 points there (Figure 5), and the fix is Day 30's: labels and fine-tuning.
- **Similarity is not meaning.** The nearest photo shares pose, background and colour as much as the object; the practice task's nearest neighbour is the *wrong* icon. Read the neighbours before trusting a score.
- **Words are ambiguous.** "boxer" the dog or the athlete, "crane" the bird or the machine: a prompt without context picks one meaning; the paper's own examples.
- **Bias travels with the weights.** CLIP's captions came from the internet with its stereotypes; the paper measures them (Section 7); a model card says what was checked.
- **Scale costs.** A million-photo gallery is a $10^6 \times 512$ table, fine; a million queries against it is 10^{12} products: libraries such as FAISS index the space.

The common mistake

Comparing *unnormalised* embeddings with a dot product, or embeddings from *two different models*: longer vectors win every search, and two models' spaces are unrelated even when both are 512 numbers. `F.normalize` both sides, the same model on both sides, and check that `S.diagonal()` reads 1.0.

What Comes Next

- **Day 32, self-study:** feature extraction against fine-tuning on one dataset, the freeze line moved one stage at a time, with the numbers.
- **Day 33, self-study:** embedding models compared (ResNet, ViT, CLIP, DINO), linear probing as a measurement of a space, CLIP zero-shot and retrieval with prompt ensembles and a vector index.
- **Day 34, Olympiad practice:** small-data fine-tuning habits: what wins points, what loses them. **Friday:** the Week 5 competition.
- **Week 6:** the same ideas for text: a sentence as an embedding, similarity between sentences, semantic search, and the transformer that CLIP's text side is made of.
- **Today's labs:** Lab 1 embeds 480 Kaggle photos of eight kinds with ResNet-18 and with CLIP, builds the similarity table, image search, a 1-NN classifier, zero-shot with prompts and a text query. Lab 2 takes 100 sports with ten labelled photos each: zero-shot against kNN against a linear probe on cached CLIP embeddings, then the submission.

Embeddings: **Summary**

Summary

- **Embedding:** a fixed-length vector that places an input so that similar inputs are close; a pretrained encoder's 512 numbers are one, for free. Closeness: Euclidean distance, dot product, **cosine similarity** $= a \cdot b/(\|a\|\|b\|)$, the angle, blind to length.
- **In code:** embed once, `F.normalize`, `S = E @ E.T`; `topk` of a row is the search result. Pixels find colours and layouts; embeddings find dogs (46.8% against 99.2% by 1-NN on 400 photos).
- **On the table:** retrieval (a query against a gallery), kNN (majority of k neighbours, no training), the linear probe on cached embeddings (Day 30's probe in seconds), and the order to try them in a contest.
- **CLIP:** an image encoder and a text encoder, 400 million pairs, a contrastive loss on the $N \times N$ table of cosines; one space for photos and sentences. Zero-shot: prompts become the head, $100\times$ cosine, softmax; 76.2% on ImageNet with no labelled photo; the prompt adds up to 5 points.
- **Where it wins and fails:** captions on the internet (cars, food, actions) against the unseen (satellite, slides, digits, signs); equal to a 4-shot probe on average, behind a 16-shot one: with labels, probe.
- **In a task:** embed, normalise, compare; use the task's structure (two partners, one of each gender) on top of the scores; validate on what you were given.

Embeddings: **References & Resources**

References & Resources

[1] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G. and Sutskever, I. (2021). Learning transferable visual models from natural language supervision. *ICML*. The 400 million pairs, the contrastive objective, Figure 1 (the pipeline), Figure 4 and Section 3.1.4 (prompt engineering, +1.3 and +3.5 points), Figure 5 (zero-shot against a ResNet-50 linear probe, 27 datasets), Figure 6 (4-shot and 16-shot), Tables 11 and 13 (ViT-B/32 zero-shot 63.2% on ImageNet; Flickr30k retrieval), Section 7 (bias).
https://arxiv.org/abs/2103.00020

[2] Dosovitskiy, A. et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*. The Vision Transformer of CLIP's image side (named here, opened in Week 6).

[3] Kornblith, S., Shlens, J. and Le, Q. (2019). Do better ImageNet models transfer better? *CVPR*. "Logistic regression on fixed features", the linear probe on cached embeddings. Kumar, A. et al. (2022). Fine-tuning can distort pretrained features. *ICLR*. LP against FT.

[4] Johnson, J., Douze, M. and Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Trans. Big Data*. FAISS, the vector index named in the limitations.

[5] Hugging Face documentation: `CLIPModel`, `CLIPProcessor`, `get_image_features`, `get_text_features`; the model card of `openai/clip-vit-base-patch32` (151 million weights, projection dimension 512, logit scale 100).
https://huggingface.co/openai/clip-vit-base-patch32 OpenAI's `clip` package, used by the IOAI 2025 baseline:
https://github.com/openai/CLIP

References & Resources (cont.)

[6] IOAI 2025 (Beijing), Individual Contest, Task 4, "Restroom Icon Matching" (the practice task; baseline 0.77, best 0.90), Task 6, "Pixel Efficiency" (a CLIP zero-shot scorer). https://github.com/IOAI-official/IOAI-2025 NEOAI 2025, Task 4, "Cluster Pictures". https://www.kaggle.com/competitions/neoai-2025-cluster-pictures

[7] Kaggle datasets through `kagglehub`: *Natural Images* (`prasunroy/natural-images`: 6,899 photos, eight classes; the deck's 400 photos and Lab 1), *100 Sports Image Classification* (`gpiosenka/sports-classification`: 13,493 training, 500 validation and 500 test photos of 224 pixels; Lab 2). Figures without another source: `slides-builder/scripts/day31_embedding_figures.py`; no model was trained for this deck, the only model runs are the published ResNet-18 and CLIP ViT-B/32 weights applied once to these photos, and every comparison number is quoted from the sources above.